Proving Basic Algorithms Correct: Sorting

Computer-Assisted Theorem Proving (7CCSACTP)

Mohammad Abdulaziz

Computation Induction

So far: *structural induction*, since all functions were primitive recursive.
But `merge` shortens `xs` in one recursive call and `ys` in the other.

Structural induction on lists: one case per constructor,
`P []` and `P xs` $\Longrightarrow$ `P (x # xs)`.

Computation induction

If `f` is defined by **fun**, Isabelle provides the rule `f.induct`:
for each defining equation `f e = ... f` r_1 `...  f` r_k `...`
prove `P e` assuming `P` $r_1, \ldots,$ `P` r_k.

One case per defining equation, one IH per recursive call.

Computation Induction: Example

```
merge [] ys = ys
merge xs [] = xs
merge (x # xs) (y # ys) =
  (if x ≤ y then x # merge xs (y # ys)
   else y # merge (x # xs) ys)
```

Computation Induction: Example

```
merge [] ys = ys
merge xs [] = xs
merge (x # xs) (y # ys) =
  (if x ≤ y then x # merge xs (y # ys)
   else y # merge (x # xs) ys)
```

Prove `length (merge xs ys) = length xs + length ys`
by `merge.induct`, with `P xs ys` this property: show

1. `P [] ys`
2. `P (x # xs) []`
3. `(x ≤ y ⟹ P xs (y # ys)) ⟹`
   `(¬ x ≤ y ⟹ P (x # xs) ys) ⟹`
   `P (x # xs) (y # ys)`

©Mohammad Abdulaziz

Computation Induction: Example

```
merge [] ys = ys
merge xs [] = xs
merge (x # xs) (y # ys) =
  (if x ≤ y then x # merge xs (y # ys)
   else y # merge (x # xs) ys)
```

Prove `length (merge xs ys) = length xs + length ys`
by `merge.induct`, with `P xs ys` this property: show

1. `P [] ys`
2. `P (x # xs) []`
3. `(x ≤ y ⟹ P xs (y # ys)) ⟹`
   `(¬ x ≤ y ⟹ P (x # xs) ys) ⟹`
   `P (x # xs) (y # ys)`

In Isabelle: `proof (induction xs ys rule: merge.induct)`
Cases are named `1`, `2`, `3`; the IH of case 3 is `"3.IH"` (with quotes).

©Mohammad Abdulaziz

Informal Proof: Length of `merge`

Lemma

```
length (merge xs ys) = length xs + length ys
```

Proof by computation induction (rule `merge.induct`), writing $|xs|$ for `length xs`:

- *Case 1:* $|\text{merge } [] \; ys| = |ys| = |[]| + |ys|$.
- *Case 2:* $|\text{merge } (x\#xs) \; []| = |x\#xs| = |x\#xs| + |[]|$.
- *Case 3,* $x \leq y$*:* $|\text{merge } (x\#xs) \; (y\#ys)| = |x\#\text{merge } xs \; (y\#ys)|$
  $= 1 + |\text{merge } xs \; (y\#ys)| = 1 + |xs| + |y\#ys|$ [IH for $x \leq y$]
  $= |x\#xs| + |y\#ys|$.
- *Case 3,* $\neg\, x \leq y$*:* symmetric, using the IH for $\neg\, x \leq y$. □

In Isabelle: `by (induction xs ys rule: merge.induct) auto`

©Mohammad Abdulaziz